

This class is contained.

<<enum>>		
Unit		
UNKNOWN	=	0
KEYWORD	=	1 << 0
STRING	=	1 << 1
COLOUR	=	1 << 2
RATIO	=	1 << 3
NUMBER	=	1 << 4
PERCENT	=	1 << 5
PX	=	1 << 6
DP	=	1 << 7
VW	=	1 << 8
VH	=	1 << 9
X	=	1 << 10
EM	=	1 << 11
REM	=	1 << 12
INCH	=	1 << 13
CM	=	1 << 14
MM	=	1 << 15
PT	=	1 << 16
PC	=	1 << 17
DEG	=	1 << 18
RAD	=	1 << 19
TRANSFORM	=	1 << 20
TRANSITION	=	1 << 21
ANIMATION	=	1 << 22
DECORATOR	=	1 << 23
FILTER	=	1 << 24
FONTEFFECT	=	1 << 25
COLORSTOPLIST	=	1 << 26
BOXSHADOWLIST	=	1 << 27
PPI_UNIT	=	INCH CM MM PT PC
LENGTH	=	PX DP VW VH EM REM PPI_UNIT
LENGTH_PERCENT	=	LENGTH PERCENT
NUMBER_PERCENT	=	NUMBER PERCENT
NUMBER_LENGTH_PERCENT	=	NUMBER LENGTH PERCENT
DP_SCALABLE_LENGTH	=	DP PPI_UNIT
ANGLE	=	DEG RAD
NUMERIC	=	NUMBER_LENGTH_PERCENT ANGLE X

+ operator|(Units lhs, Units rhs) : Units
+ operator&(Units lhs, Units rhs) : Units
+ Any(Units units) : bool

```
1 enum class Unit {  
2     UNKNOWN = 0,
```

```

3
4 // Basic types.
5 KEYWORD = 1 << 0, // generic keyword; fetch as <int>
6 STRING = 1 << 1, // generic string; fetch as <String>
7 COLOUR = 1 << 2, // colour; fetch as <Colourb>
8 RATIO = 1 << 3, // ratio defined as x/y; fetch as <Vector2f>
9
10 // Numeric values.
11 NUMBER = 1 << 4, // number unsuffixed; fetch as <float>
12 PERCENT = 1 << 5, // number suffixed by '%'; fetch as <float>
13 PX = 1 << 6, // number suffixed by 'px'; fetch as <float>
14
15 // Context-relative values.
16 DP = 1 << 7, // density-independent pixel; number suffixed by 'dp';
17 ↪ fetch as <float>
18 VW = 1 << 8, // viewport-width percentage; number suffixed by 'vw';
19 ↪ fetch as <float>
20 VH = 1 << 9, // viewport-height percentage; number suffixed by 'vh';
21 ↪ fetch as <float>
22 X = 1 << 10, // dots per px unit; number suffixed by 'x'; fetch as
23 ↪ <float>
24
25 // Font-relative values.
26 EM = 1 << 11, // number suffixed by 'em'; fetch as <float>
27 REM = 1 << 12, // number suffixed by 'rem'; fetch as <float>
28
29 // Values based on pixels-per-inch.
30 INCH = 1 << 13, // number suffixed by 'in'; fetch as <float>
31 CM = 1 << 14, // number suffixed by 'cm'; fetch as <float>
32 MM = 1 << 15, // number suffixed by 'mm'; fetch as <float>
33 PT = 1 << 16, // number suffixed by 'pt'; fetch as <float>
34 PC = 1 << 17, // number suffixed by 'pc'; fetch as <float>
35 PPI_UNIT = INCH | CM | MM | PT | PC,
36
37 // Angles.
38 DEG = 1 << 18, // number suffixed by 'deg'; fetch as <float>
39 RAD = 1 << 19, // number suffixed by 'rad'; fetch as <float>
40
41 // Values tied to specific types.
42 TRANSFORM = 1 << 20, // transform; fetch as <TransformPtr>, may
43 ↪ be empty
44 TRANSITION = 1 << 21, // transition; fetch as <TransitionList>
45 ANIMATION = 1 << 22, // animation; fetch as <AnimationList>
46 DECORATOR = 1 << 23, // decorator; fetch as <DecoratorsPtr>
47 FILTER = 1 << 24, // decorator; fetch as <FiltersPtr>
48 FONTEFFECT = 1 << 25, // font-effect; fetch as <FontEffectsPtr>
49 COLORSTOPLIST = 1 << 26, // color stop list; fetch as
50 ↪ <ColorStopList>
51 BOXSHADOWLIST = 1 << 27, // shadow list; fetch as <BoxShadowList>
52
53 LENGTH = PX | DP | VW | VH | EM | REM | PPI_UNIT,

```

```

48     LENGTH_PERCENT = LENGTH | PERCENT,
49     NUMBER_PERCENT = NUMBER | PERCENT,
50     NUMBER_LENGTH_PERCENT = NUMBER | LENGTH | PERCENT,
51     DP_SCALABLE_LENGTH = DP | PPI_UNIT,
52     ANGLE = DEG | RAD,
53     NUMERIC = NUMBER_LENGTH_PERCENT | ANGLE | X
54 };
55
56 // Type alias hint for indicating that the value can contain multiple
57 ↳ units OR-ed together.
58 using Units = Unit;
59
60 // OR operator for combining multiple units where applicable.
61 inline Units operator|(Units lhs, Units rhs)
62 {
63     using underlying_t = std::underlying_type<Units>::type;
64     return static_cast<Units>(static_cast<underlying_t>(lhs) |
65                               ↳ static_cast<underlying_t>(rhs));
66 }
67 // AND operator for combining multiple units where applicable.
68 inline Units operator&(Units lhs, Units rhs)
69 {
70     using underlying_t = std::underlying_type<Units>::type;
71     return static_cast<Units>(static_cast<underlying_t>(lhs) &
72                               ↳ static_cast<underlying_t>(rhs));
73 }
74
75 // Returns true if any unit is set.
76 inline bool Any(Units units)
77 {
78     return units != Unit::UNKNOWN;
79 }

```
